



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI



sAifer Lab

Joint lab on Safety and Security of AI

Il World Wide Web

Maura Pintor

maura.pintor@unica.it

In questa lezione

Panoramica sul Web e come funziona

- Client e server
- Il ruolo dei browser

Anatomia di un server web

- Cosa è un server web
- Gestione delle richieste

Dialogo tra client e server e RESTful API

- Il protocollo HTTP
- Metodi HTTP (GET, POST, PUT, DELETE)
- Application Programming Interfaces (APIs)

Sessioni Web

- Cosa sono le sessioni e perché servono
- HTTP stateless vs sessioni stateful
- Cookie e gestione delle sessioni
- Sicurezza delle sessioni
- Autenticazione e autorizzazione



Il World Wide Web (WWW)

1989 Il WWW fu proposto al CERN di Ginevra da Tim Berners-Lee

1990 Con Robert Cailliau proposero l'architettura client-server

1991 Primo sito Web (solo per la comunità scientifica)

1993 La tecnologia alla base di Web fu resa pubblica

1996 HTTP/1.0

1999 HTTP/1.1

2015 HTTP/2



Il logo del WWW disegnato da [Robert Cailliau](#)

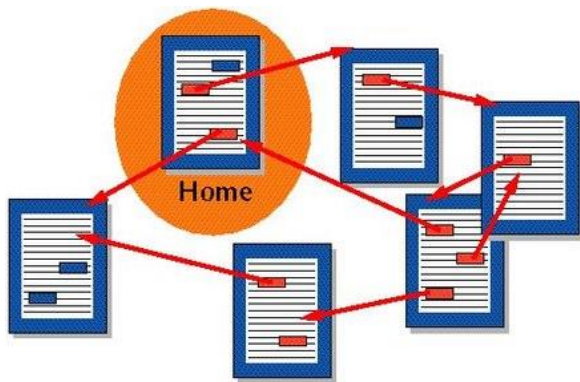
Ipertesti e Web

Un **Hypertext** (ipertesto) è un insieme di documenti collegati tramite **link** (collegamenti)

I documenti nel Web sono di solito formattati con l'Hypertext Markup Language (**HTML**)

Il testo **non è più lineare**, ma può essere interconnesso a formare un **grafo**

Si possono inserire anche contenuti multimediali quali video, immagini, audio



Il World Wide Web (WWW)

Il WWW è quindi una ragnatela (Web) di documenti interconnessi che sono messi a disposizione da computer geograficamente distribuiti (World Wide)

I documenti sono localizzati tramite degli **identificatori virtuali**

L'accesso ai documenti è regolato tramite appositi **protocolli**

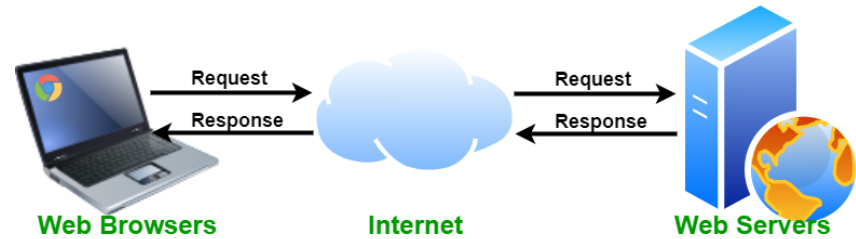


Componenti fisici del WWW

Abbiamo a che fare ogni giorno con dei **server** e **client**

Esempio di web server: <https://en.wikipedia.org/wiki/>

server: programma applicativo che accetta connessioni e risponde a specifiche richieste



client: programma applicativo che stabilisce una connessione al fine di inviare delle richieste

Identificatori Virtuali

Tutte le risorse Web sono identificate tramite degli identificatori univoci chiamati **Uniform Resource Identifier (URL)**

Gli URI identificano la risorsa (documento, immagine, servizio, ...) a cui vogliamo accedere

Gli URI seguono una sintassi standard (come specificato nel nome, Uniform)

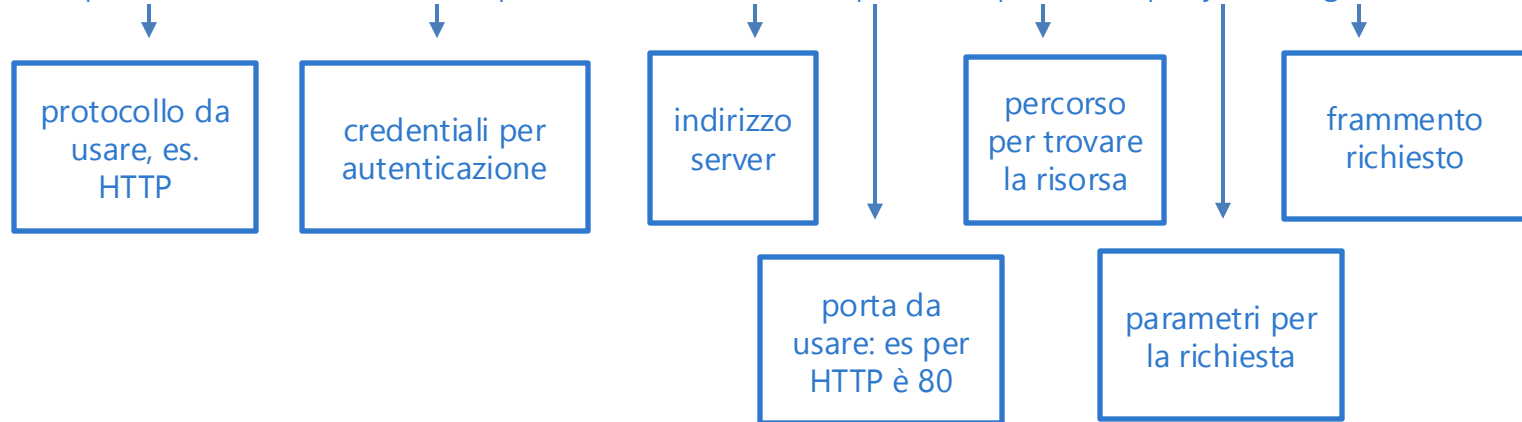
<schema> : <parte specifica>

Specializzazioni di URI: URL e URN

Ci sono due specializzazioni di URI:

- **Uniform Resource Name (URN)** identifica una risorsa tramite un nome globalmente unico
 - esempio: (International Standard Book Number (ISBN))
- **Uniform Resource Locator (URL)** identifica una risorsa tramite il meccanismo di accesso primario (per esempio la sua localizzazione nella rete o il protocollo necessario)
 - esempio: schema http-like

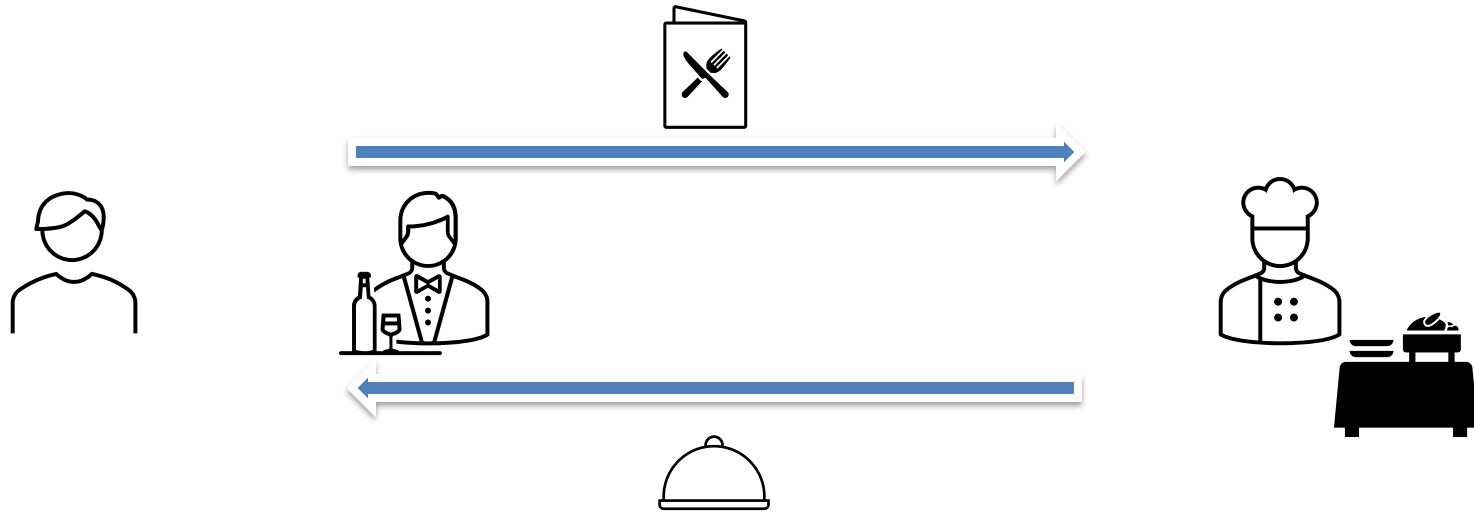
`<protocol>://[<username>:<password>@]<host>[:<port>][<path>[?<query>][#fragment]`



Le Application Programming Interfaces (API)

Le API sono un set di regole e protocolli per interagire con applicazioni software

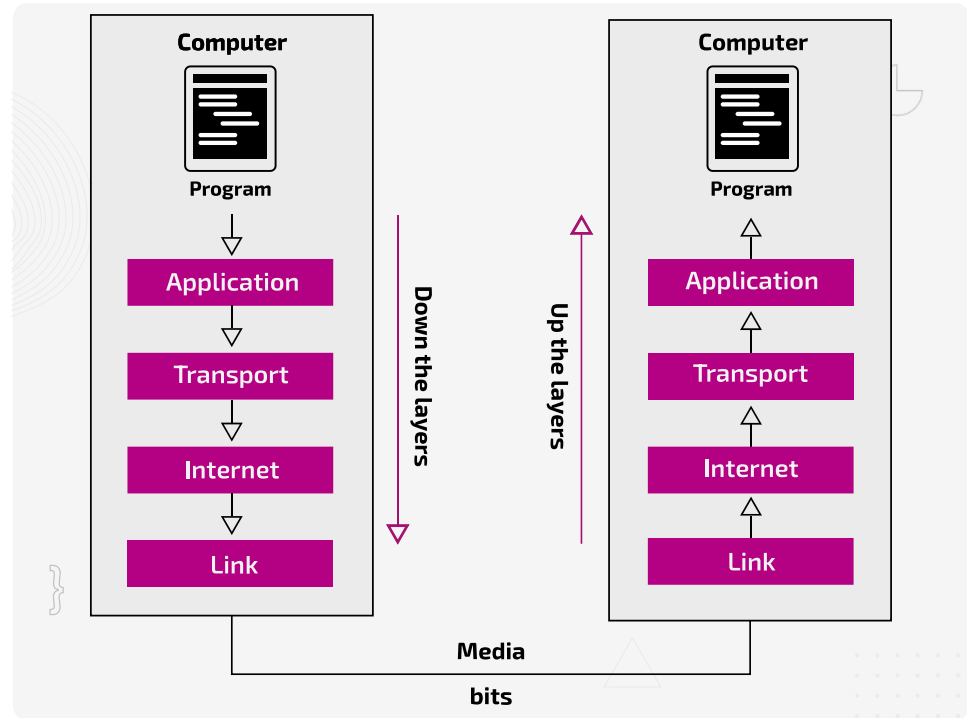
Le API hanno un ruolo chiave nello sviluppo, in quanto permettono di far comunicare componenti indipendenti (inclusi servizi esterni)



HyperText Transfer Protocol (HTTP)

L' HyperText Transfer Protocol (HTTP) è il protocollo di **livello applicativo dello stack TCP/IP** usato per trasferire contenuti Web

È un protocollo **stateless**: né il server né il client mantengono informazioni sullo storico di messaggi precedentemente scambiati



Messaggi HTTP

I messaggi HTTP sono sequenze di byte concettualmente atomiche

Sono composti da due parti

- **header**: contiene informazioni per la gestione della comunicazione e caratteristiche del messaggio
- **body**: contiene i dati trasportati dal messaggio

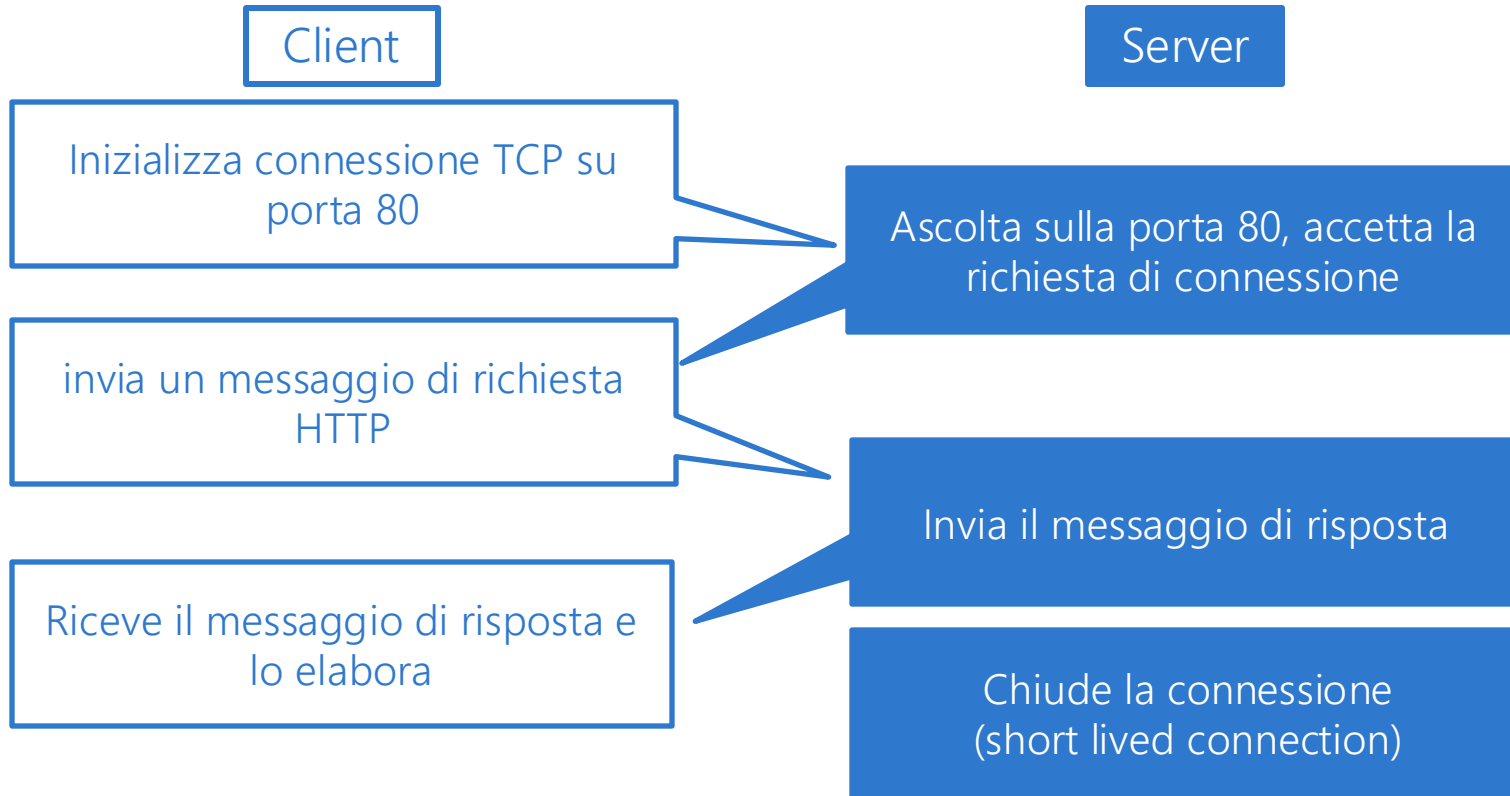
A livello di trasporto, sono gestiti con TCP (UDP non è adatto perché non garantisce la consegna)

Il Server (passivo) rimane in ascolto, tipicamente sulla porta 80

Il Client (attivo) invia una richiesta, aprendo una connessione sulla porta del server

Il server accetta la connessione, manda la risposta, e chiude la connessione

Esempio di comunicazione client-server (HTTP/1.0)



HTTP/1.1

In HTTP/1.1 la stessa connessione può restare attiva per più richieste (connessione persistente)

Il server chiude la connessione quando:

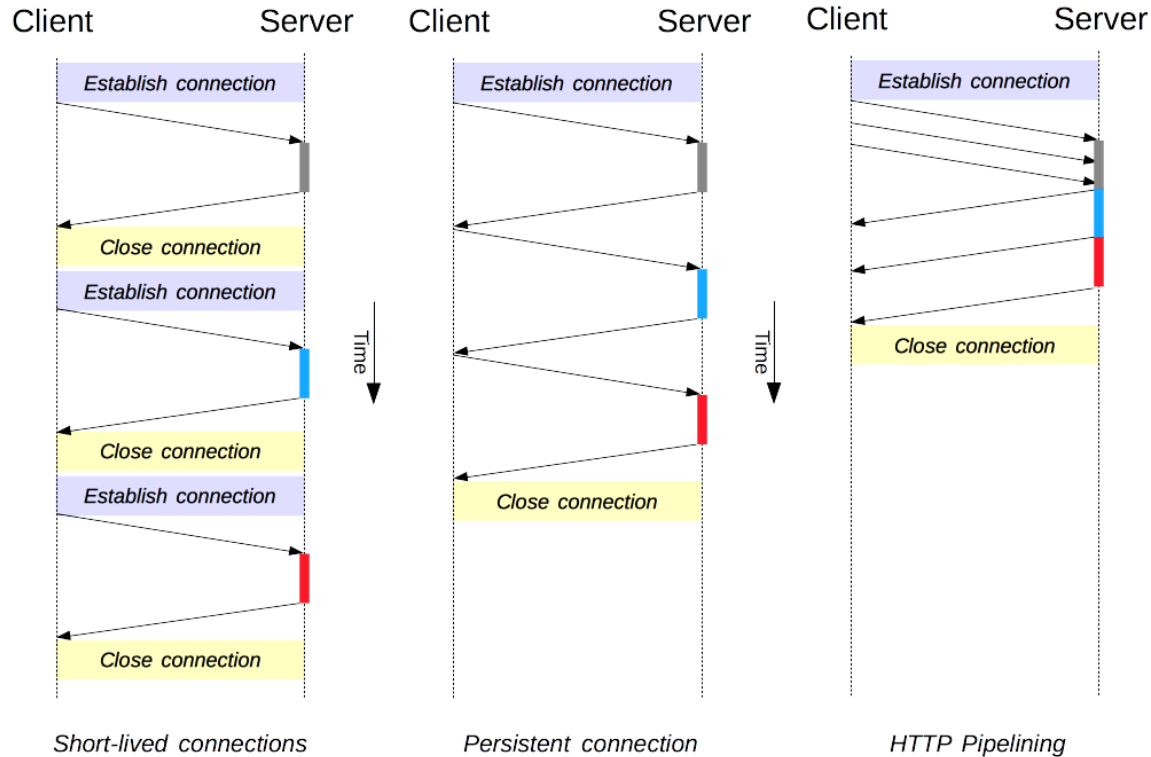
- il client manda un messaggio che richiede la chiusura
- o quando la connessione non è usata per un certo tempo (timeout)

I messaggi possono essere mandati anche tramite **pipelining**

- richieste multiple inviate anche prima il termine della ricezione delle risposte
- le risposte sono date nello stesso ordine delle richieste



Tipi di connessione



HTTP 2.0

Miglioramento prestazioni e mantenimento dei precedenti standard:

- backward compatibility, consente di usare anche i precedenti HTTP/1.0 e HTTP/1.1
- compressione dei dati degli header HTTP
- tecnologie push lato server (inviare più dati rispetto a quelli richiesti dal client)
- caricamento parallelo degli elementi su una singola connessione TCP (multiplexing)

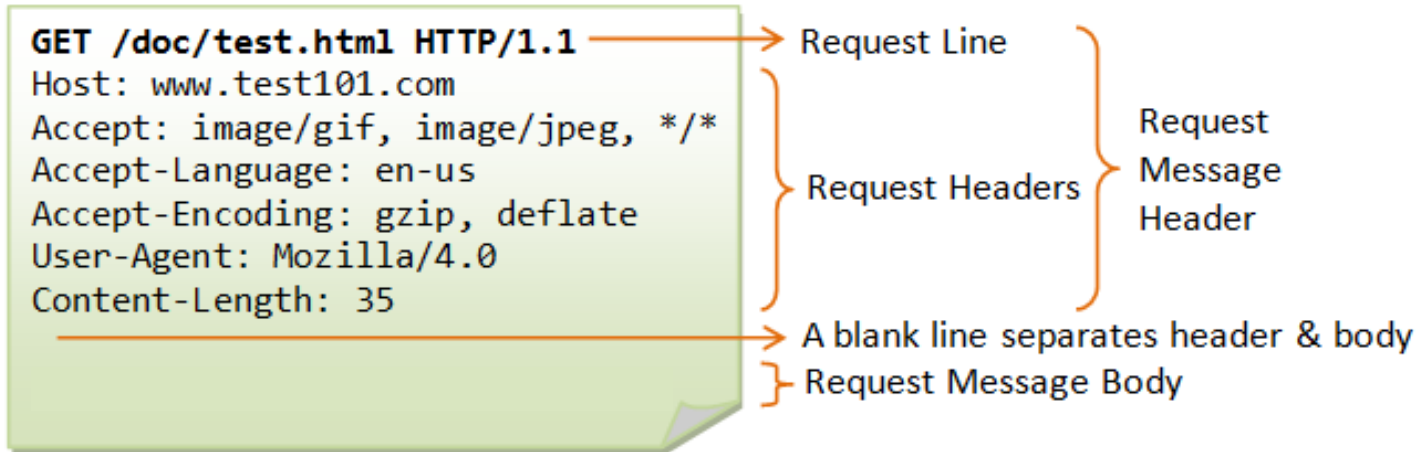


Esempio di richiesta HTTP

L'header specifica i comandi (GET, POST, ...), l'oggetto, e la versione di protocollo
GET è uno dei **metodi** (comandi) HTTP usato per richiedere una risorsa a un server

Per questo tipo di richiesta di solito il message body è vuoto

Il passaggio di parametri di solito avviene tramite la parte <query> dell'URL



Metodi HTTP più usati

GET

- serve per chiedere risorse
- passaggio di parametri tramite URL <query>
- la lunghezza dell'URL è limitata

POST

- i dettagli della risorsa sono contenuti nel body del messaggio
- non ha limiti di lunghezza per i parametri
- il message body può essere *cifrato*

Altri metodi HTTP

PUT

- Chiede al server di *memorizzare* una risorsa all'URL specificato
- Causa la creazione/sostituzione di una risorsa (contrariamente agli altri metodi)

DELETE

- Richiede la cancellazione di una risorsa

HEAD

- Simile al metodo GET, ma il server deve rispondere solo con l'header
- Usato per verificare se un URL *esiste* o se è *accessibile* senza autenticazione

OPTIONS

- richiede informazioni sulle opzioni disponibili per la comunicazione

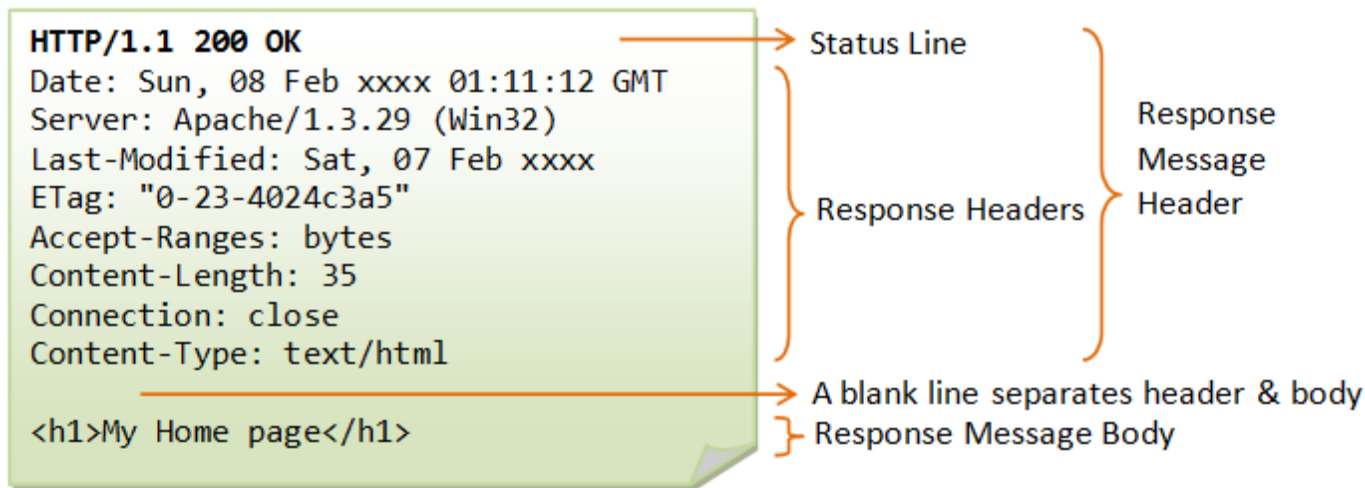
TRACE

- usato prevalentemente per il testing dei servizi Web, serve a chiedere al server cosa ha ricevuto

Esempio di risposta HTTP

La risposta include uno status code che indica se la richiesta è andata a buon fine o se ci sono stati problemi

Se la risposta è andata a buon fine, nel message body è inclusa la risorsa



Status code HTTP

Lo status code è composto da 3 cifre, la prima indica la classe di risposta e le altre due l'errore specifico

- **1xx: informational.** Risposta temporanea alla richiesta, comunque usata poco dopo HTTP/1.0.
- **2xx: success.** La richiesta è stata processata correttamente.
 - esempio: 200 OK
- **3xx: redirect.** Il server ha ricevuto la richiesta, ma il client deve eseguire altre azioni per portarla a termine.
 - esempio: 301 Moved permanently (ma il server conosce la nuova posizione)
- **4xx: client error.** La richiesta non è formattata correttamente o non è autorizzata
 - esempio: 404 Not found (url errato)
- **5xx: server error.** La richiesta può essere corretta, ma il server ha un problema interno per cui non è in grado di rispondere.
 - esempio: 500 Internal server error

I cookie HTTP

I cookie sono piccoli blocchi di dati (collezioni di stringhe) creati da un web server e salvati nel browser del client

Tramite i cookie, i web server possono stabilire una comunicazione **stateful** (per esempio i prodotti aggiunti nel carrello in un negozio online)

Dopo la loro creazione, i cookies vengono continuamente passati a ogni trasmissione

Autenticazione e autorizzazione

Ci sono casi in cui le risorse hanno accesso ristretto solo agli utenti abilitati

Tipicamente si usano le seguenti tecniche

- Filtro su set di indirizzi IP
 - sconsigliata, presenta problemi con IP dinamici (DHCP), e di falsificazione degli IP (spoofing)
- HTTP Digest (invio di password dopo hash)
 - in disuso
- **HTTP Basic Access authentication (BA)**
 - il server chiede l'autenticazione e il client risponde con username e password codificati in Base64
 - non prevede protezione per le credenziali trasmesse (non cifrate!)
- **Form per la richiesta di username e password**
 - normalmente si usa il metodo POST per inviare le credenziali



Sicurezza dell'autenticazione

Il Secure Sockets Layer (SSL) è nato come protocollo di comunicazione **sicuro** per proteggere le comunicazioni sul Web

- Consentiva crittografia e autenticazione

Il Transport Layer Security (TLS) ha sostituito SSL nel 1999 dopo varie iterazioni

TLS e SSL utilizzano entrambi **certificati digitali** (chiave privata e chiave pubblica) che facilitano il processo di handshake e stabiliscono comunicazioni crittografate tra un browser e un server Web

HTTPS è la pratica di stabilire un protocollo SSL/TLS sicuro su una connessione HTTP non sicura